WEEK พิเศษ · ต่อจาก LAB 7.2

Maze Pilot Challenge

ไม่ต้องเทรน ทุกทีมใช้ policy ตัวเดียวกันที่แจกไป แข่งกันที่สมองของหุ่น: pilot.py ที่ตัดสินใจว่าจะเดินทางไหน เร็วแค่ไหน ส่งเข้าเครื่องครูแล้ววิ่งจับเวลาบนจอใหญ่

วันนี้ทำอะไร

1 คาบ 3 ช่วง ส่งได้ 2 ไฟล์

~10 นาที

1 · ดูโจทย์

ด่านมีทางสั้นแต่ยาก กับทางยาวแต่ง่าย pilot เริ่มต้นเลือกทางไม่เป็น

~30 นาที

2 · แก้ pilot + ซ้อม

ทำตามคำใบ้ทีละขั้น ลองในเครื่องตัวเอง แล้วส่งเข้ารอบซ้อม (ด่าน seed 1)

~20 นาที

3 · แข่งจริง

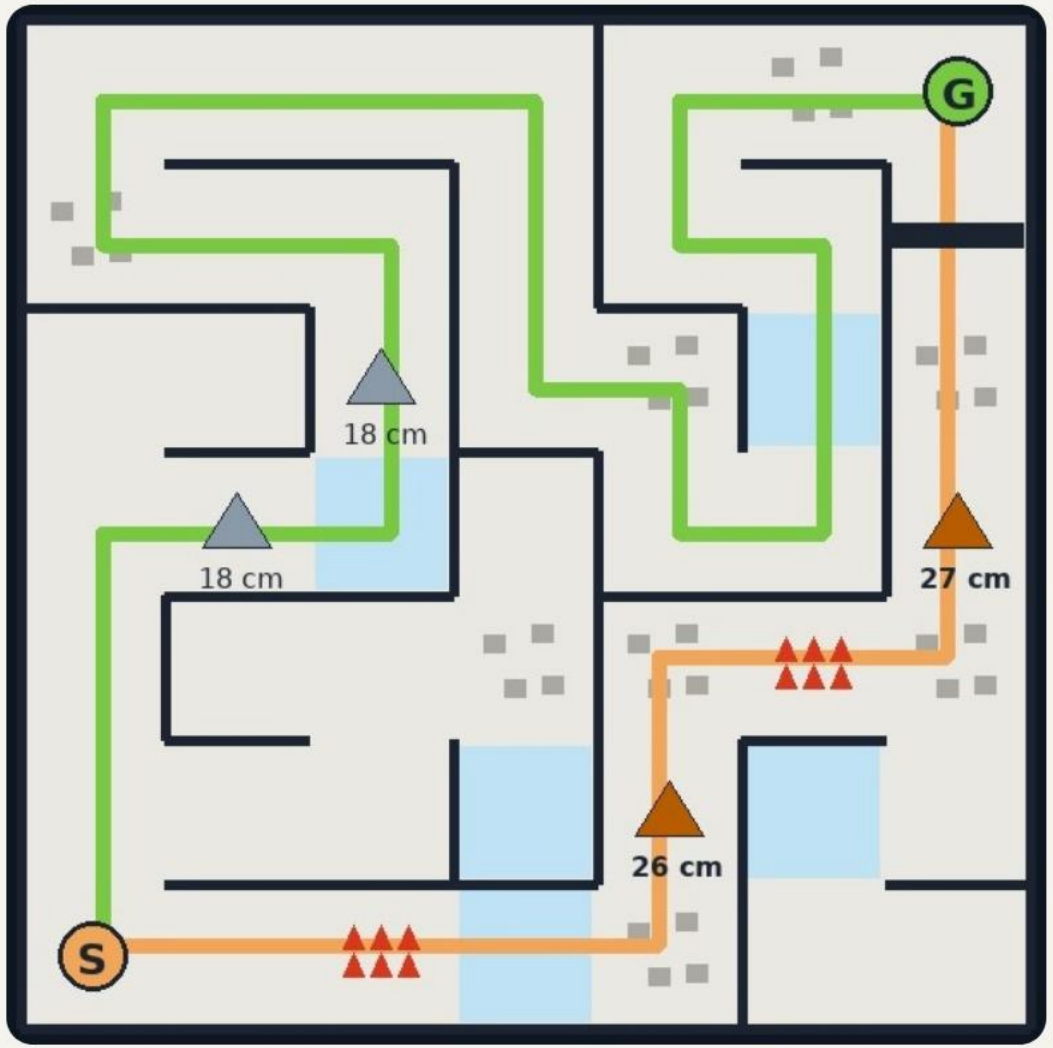
ครูประกาศ seed ใหม่ ปิดรับงาน แล้วรันทุกทีมขึ้นจอ

`policy.pt` · ไม่ต้องแก้

ใช้ไฟล์ที่แจกไป pretrained/g1_rough/model_pretrained.pt ทุกทีมเหมือนกัน

`pilot.py` · ตัวที่แข่งกัน

คัดลอก DefaultPilot จาก scripts/challenge.py มาเป็นจุดเริ่ม แล้วทำให้ฉลาดขึ้น



ตัวอย่างด่าน seed 7: ส้ม = ทางสั้น เขียว = ทางยาว สามเหลี่ยม = เนิน (น้ำตาล = ชัน) แดง = หนาม ฟ้า = น้ำแข็ง

กติกาย่อ

ทางสั้นไม่ได้แปลว่าเร็ว

ด่าน	7 × 7 ช่อง (14 × 14 m) ออกจากจุดเริ่มได้ 2 ทาง ด่านสุ่มจาก seed
ทางสั้น	ขรุขระ ร่อง **เนินชัน 24–30 cm ที่ policy ที่แจกปีนไม่ขึ้น** หนาม 2 แปลง
ทางยาว	เลี้ยวเยอะกว่า แต่เนินเตี้ย 10–18 cm ปีนสบาย มีน้ำแข็งกับของเกะกะ
ล้ม / ค้าง	กลับไปจุดเริ่ม **เวลายังเดินต่อ** (ค้าง = ขยับไม่ถึง 0.25 m ใน 10 s)
อันดับ	ถึงเร็วสุดชนะ ไม่ถึงใน 300 s ดูระยะที่เหลือน้อยสุด แล้วดูล้มน้อยกว่า
ห้าม	อ่านแผนที่จริง (GENERATED, HAZARDS) หรือเขียนไฟล์ไว้ใช้รอบหน้า ใช้ได้แค่สิ่งที่หุ่นเห็น

PILOT.PY

จุดเริ่ม: pilot ของ Lab 7.2

```python
from g1_maze_lab.autopilot import MazeAutopilot

class Pilot: # ชื่อนี้เท่านั้น
    def __init__(self, num_envs, num_cells, cell_size, device):
        self.ap = MazeAutopilot(num_envs, num_cells, cell_size,
            device, strategy="flood", speed=0.6,
            turn_early=0.6, min_speed=0.3, max_turn=1.0)

    def act(self, obs): # ทุก 0.02 s
        ap, p = self.ap, obs["pos_xy"]
        walls = ap.walls_from_height_scan(
            obs["scan_xy"], obs["scan_z"], ap.cell_of(p))
        return ap.step(p, obs["yaw"], walls, obs["dt"])

    def reset(self, env_ids): # หลังล้ม
        self.ap.reset(env_ids)
```

obs[...]	คืออะไร
pos_xy, yaw	ตำแหน่งเทียบจุดเริ่ม (m) และทิศที่หัน (rad)
scan_xy, scan_z	height scan 33 × 33 จุด ห่างกัน 0.1 m รอบตัว
lidar_dist	lidar 72 เส้น ระดับอก
lin_vel_b	ความเร็วจริงของลำตัว
t, dt	เวลาที่ผ่านไป และ 0.02 s

return

คำสั่งความเร็ว (N, 3)

v_x, v_y (m/s) และ w_z (rad/s) เทียบกับตัวหุ่น policy แปลงเป็นท่าเดินเอง

ลองในเครื่องตัวเอง

แก้ 1 อย่าง รัน 1 รอบ ดูผล

```
# ในโฟลเดอร์ g1_maze_lab · ไม่ใส่ --policy = ใช้ Pretrained
$ uv run python scripts/challenge.py --pilot pilot.py --seed 7 --viz newton

# ไม่เปิดหน้าต่าง เร็วกว่า ดูแค่ผลกับจุดที่ล้ม
$ uv run python scripts/challenge.py --pilot pilot.py --seed 7 \
    2>&1 | grep -E "fall|stuck|GOAL|RESULT"
[CHALLENGE] t=21.4s fall/crash #1 in cell (2, 0) → back to start
[CHALLENGE] RESULT team=(test) seed=7: ...
```

ลองหลาย seed

seed 7, 11 ทางสั้นยาก · seed 13, 15 ทางสั้นอยู่อีกฝั่ง ตอนแข่งครูประกาศ seed ใหม่ที่ยังไม่มีใครเห็น

รอนานไป?

ใส่ CHALLENGE_TIME=120 หน้าคำสั่ง ตัดรอบที่ 120 วินาที พอเห็นว่าดีขึ้นไหม

ผลแกว่ง

ฟิสิกส์ไม่ได้ออกเหมือนเดิมทุกรอบ ดีขึ้นจริงต้องดีขึ้นซ้ำ ๆ ไม่ใช่รอบเดียว

คำใบ้ 4 ขั้น

ทำให้หุ่นไปได้ไกลขึ้น ทีละขั้น

ขั้น 1 · ง่าย

ดูว่าพังตรงไหน

อ่าน log ช่องที่ล้ม ก่อนแก้อะไรทั้งนั้น

ขั้น 2 · ง่าย

ปรับปุ่มความเร็ว

แก้ตัวเลข 4 ตัวใน MazeAutopilot ได้เวลาดีขึ้นนิดหน่อย

ขั้น 3 · กลาง · คุ้มสุด

จำช่องที่ล้ม

ล้มที่เดิม 2 ครั้ง = ถือว่าเป็นกำแพง แล้วไปทางอื่น จากไปไม่ถึงกลายเป็นถึงเส้นชัย

ขั้น 4 · ยาก

เห็นก่อนล้ม

อ่าน height scan หาเนินชันกับหนาม เลี่ยงตั้งแต่ยังไม่เหยียบ

ทำไมขั้น 3 สำคัญ: policy ที่แจกปืนเนินชันบนทางสั้นไม่ขึ้น ถ้า pilot ไม่ยอมเปลี่ยนทาง ปรับความเร็วยังไงก็ไปไม่ถึง

ขั้น 1 · ดูว่าพังตรงไหน

pilot เดิมล้มที่ช่องเดิมซ้ำ ๆ

```
# pilot เริ่มต้น ด่าน seed 7
t=7.4s fall/crash #1 in cell (3, 0)
t=15.2s fall/crash #2 in cell (2, 0)
t=22.8s fall/crash #3 in cell (3, 0)
t=29.3s fall/crash #4 in cell (2, 0)
t=37.3s fall/crash #5 in cell (3, 0)
... ล้มเกือบ 30 ครั้ง ไม่ถึงเส้นชัย
```

ทำไมถึงเข้าทางเดิม

flood fill เลือกทางที่ **สั้นที่สุด** เท่าที่รู้ ล้มแล้ว autopilot ลืมแผนที่ เริ่มใหม่ก็เลือกทางสั้นเดิมอีก มันไม่รู้ว่าทางนั้น **ยาก**



รันแบบ --viz newton ดูด้วยตาว่าล้มเพราะอะไร: ร่อง หนาม เนินน้ำแข็ง หรือชนกำแพง

ขั้น 2 · ปรับปุ่มความเร็ว

แก้ตัวเลขบรรทัดเดียว เร็วขึ้นราว 15%

```
self.ap = MazeAutopilot(num_envs, num_cells, cell_size,
    device, strategy="flood",
    speed=0.8,       # เดิม 0.6 m/s ทางตรง
    turn_early=0.6, # เริ่มเลี้ยวก่อนถึงกลางช่อง (m)
    min_speed=0.3,  # ความเร็วตอนเลี้ยว
    max_turn=1.0)   # หมุนตัวได้เร็วสุด (rad/s)
```

seed	speed 0.6 (เดิม)	speed 0.8
13	ถึง 73 s	**ถึง 61 s**
15	ถึง 89 s	**ถึง 75 s**
7	ไม่ถึง ล้ม 27	ถึง 164 s ล้ม 11

เร็วขึ้น = ล้มง่ายขึ้น

เร็ว 0.8 + ขั้น 3: seed 7 ถึง 130 s แต่ seed 11 กับ 5 ไม่ถึง (ตอน 0.6 ถึงทั้งคู่) ด่านง่ายเร่งได้ ด่านยากควรช้า รันหลาย seed ก่อนเชื่อ

ลองต่อเอง

ช้าลงบนน้ำแข็ง: เทียบ obs["lin_vel_b"] กับที่สั่ง ถ้าต่างกันมาก แปลว่ากำลังลื่น

ขั้น 3 · จำช่องที่ล้ม

ล้มที่เดิม 2 ครั้ง = กำแพง

```python
def act(self, obs):
    cell = self.ap.cell_of(obs["pos_xy"])
    self.last_cell[:] = cell # ตอนนี้อยู่ช่องไหน
    walls = ...walls_from_height_scan(...)
    # TODO ก: ด้านที่ติดช่อง blocked ให้เป็นกำแพง
    return self.ap.step(...)

def reset(self, env_ids): # ถูกเรียกหลังล้ม
    self.ap.reset(env_ids) # autopilot ลืมแผนที่
    for e in torch.as_tensor(env_ids).tolist():
        i, j = self.last_cell[e].tolist()
        self.falls[e, i, j] += 1
        if self.falls[e, i, j] ⩾ 2:
            self.blocked[e, i, j] = True
        # TODO ข: ใส่ 1 ใน self.ap.walls ทั้ง 4 ด้าน
        # ของทุกช่องที่ blocked (และฝั่งช่องข้าง ๆ)
```

กับดัก 3 อย่าง

1. ap.reset ลบกำแพงทุกครั้ง ต้องใส่ช่อง blocked กลับทุกรอบ

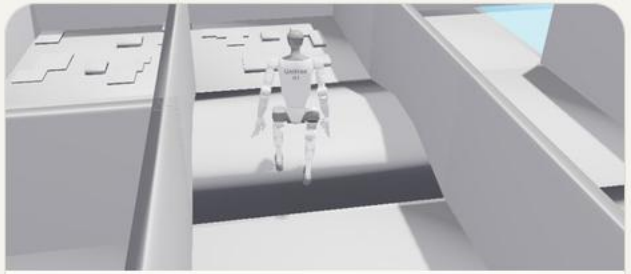
2. scan เห็นว่าด้านนั้นโล่ง autopilot จะเปิดกลับ (TODO ก)

3. ห้ามปิดทางสุดท้าย: ถ้า ap.flood_fill บอกว่าไปไม่ถึงแล้ว ให้ยกเลิกช่องนั้น

seed	pilot เดิม	จำช่องที่ล้ม
7	ไม่ถึง (เหลือ 6.5 m)	**ถึง 149 s**
11	ไม่ถึง (เหลือ 6.4 m)	**ถึง 196 s**
5	ไม่ถึง (เหลือ 30.4 m)	**ถึง 188 s**
13	ถึง 73 s	ถึง 73 s
15	ถึง 89 s	ถึง 87 s

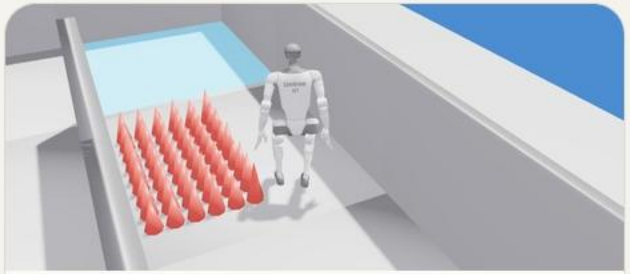
ขั้น 4 · เห็นก่อนล้ม

อ่าน scan_z แยกเนินชันกับหนาม



เนินชัน: สูงแต่เรียบ

สูงเกิน 21 cm และจุดข้าง ๆ ต่างกันไม่ถึง 10 cm → ทั้งช่องเป็นกำแพง



หนาม: ขรุขระแหลม

จุดติดกันสูงต่างกันเกิน 10 cm → เป็นกำแพง หรือ (ยากกว่า) เดินเลียบฝั่งที่โล่ง

```
# สูงสุด / ต่ำสุดในหน้าต่าง 3 × 3 รอบแต่ละจุด
g = obs["scan_z"].reshape(-1, 1, 33, 33)
hi = F.max_pool2d(g, 3, 1, 1)
lo = -F.max_pool2d(-g, 3, 1, 1)
bumpy = hi - lo > 0.10
steep = (g > 0.21) & ~bumpy & (hi < 0.45)
spiky = bumpy & (hi < 0.45) & (lo > -0.05)
# จุดไหนอยู่ช่องไหน: ap.cell_of(scan_xy)
# นับต่อช่อง เกิน ~15 จุด = เชื่อได้
```

ดูเนินอย่างเดียวไม่พอ

seed 7 หนามอยู่ก่อนเนิน หุ่นล้มที่หนามก่อนเห็นเนิน: ไม่ถึง ล้ม 27 ครั้ง · seed 11 เห็นเนินก่อน: ถึง 122 s ไม่ล้มเลย

เนิน + หนาม + ขั้น 3

เฉลยครู (เนิน + เดินเลียบหนาม): seed 7 ถึง 138 s ไม่ล้ม seed 11 ถึง 122 s ใส่ขั้น 3 ไว้เป็นตาข่ายกันพลาดด้วยก็ได้

ส่งงาน

ส่งเข้าเครื่องครู ผลขึ้นบนหน้าเว็บ

1. ต่อ Wi-Fi วงเดียวกับครู เปิด http://<IP เครื่องครู>:8000
2. **ชื่อทีม** a-z 0-9 _ - เช่น team_tiger ใช้ชื่อเดิมทุกครั้ง
3. **policy.pt** เลือกไฟล์ pretrained/g1_rough/model_pretrained.pt
4. **pilot.py** ของทีม (+ ไฟล์ .py ที่ import ได้อีก 5 ไฟล์)
5. กดส่ง กรอบแดงบอกว่าผิดตรงไหน แก้แล้วส่งใหม่ได้เรื่อย ๆ

รอบซ้อม · seed 1

ส่งเมื่อไหร่ เครื่องครูรันให้ตามคิว ดูสถานะ รอรัน → กำลังรัน → รันแล้ว พร้อมเวลา

แข่งจริง · seed ใหม่

ครูประกาศ seed ปิดรับงาน แล้วรันทุกทีมขึ้นจอ ทีมละ 2 รอบ เอารอบที่ดีกว่า

ส่งรอบซ้อมสักครั้งก่อนแข่ง จะได้รู้ว่าไฟล์ผ่าน และ pilot ไม่พังบนเครื่องครู ส่งซ้ำระหว่างที่ทีมเรากำลังวิ่งไม่ได้



วิธีส่ง · ทีละขั้น

กรอกฟอร์มแบบนี้

0. ต่อ Wi-Fi เดียวกับครู เปิดเบราว์เซอร์ไปที่ http://<IP ครู>:8000

1. **ชื่อทีม** อังกฤษ ตัวเลข _ - เช่น team_tiger ส่งครั้งต่อไปใช้ชื่อเดิม (ไฟล์ใหม่ทับไฟล์เก่า)
2. **policy.pt** กด Browse… ไปที่โฟลเดอร์ g1_maze_lab/pretrained/g1_rough/ เลือก model_pretrained.pt (7.8 MB)
3. **pilot.py** ไฟล์ของทีม ชื่อไฟล์อะไรก็ได้ ระบบเปลี่ยนเป็น pilot.py ให้
4. **ไฟล์ .py ที่ pilot ใช้** เว้นว่างได้ ใช้เมื่อแยกโค้ดหลายไฟล์
5. กดปุ่มเขียว **ส่งงาน** แล้วดูกรอบด้านบนของหน้า (หน้าถัดไป)

วิธีส่ง · กดแล้วเกิดอะไร

ดูกรอบด้านบน แล้วเลื่อนลงดูตาราง

กรอบแดง = ยังไม่ได้ส่ง

อ่านว่าผิดตรงไหน (ตัวอย่าง: pilot.py บรรทัด 2 ลืม :) แก้แล้วกดส่งใหม่

✗ pilot.py บรรทัด 2: expected ':'

กรอบเขียว = ส่งแล้ว

บอกชื่อทีมและไฟล์ที่ระบบได้รับ

✓ ส่งงานทีม **team_tiger** แล้ว: policy.pt, pilot.py

รอครูรัน ผลจะขึ้นในตารางด้านล่าง

รอคิว

ตาราง “อันดับตอนนี้” ท้ายหน้า บอกว่าทีมเราคิวที่เท่าไร หน้าอัปเดตเอง

#	ทีม	ผล	ล้ม	รอบที่รัน	สถานะ
–	team_tiger	ยังไม่มีผล		0	คิวที่ 1

รันแล้ว

เวลาที่ได้ ล้มกี่ครั้ง ถ้าขึ้น error จะบอกบรรทัดที่พัง

#	ทีม	ผล	ล้ม	รอบที่รัน	สถานะ
1	team_tiger	96.1 s	1	1	รันแล้ว

สำหรับครู

เปิดเครื่องครู 2 เทอร์มินัล

```
# รอบซ้อม: รันทุกงานที่ส่งเข้ามาทันที
$ CHALLENGE_SEED=1 uv run python scripts/challenge_server.py
$ uv run python scripts/challenge_runner.py --seed 1 --watch --no-replay

# แข่งจริง: Ctrl+C ทั้งสองตัว แล้วเปิดใหม่ด้วย seed ที่ประกาศ · /board ขึ้นโปรเจกเตอร์
$ CHALLENGE_SEED=7 uv run python scripts/challenge_server.py
$ uv run python scripts/challenge_runner.py --seed 7 # รอบ 2: --all
```

คิว

ทีมละ 1.5–3 นาที 8 ทีม ≈ 20 นาที ทำเป็นกลุ่ม ไม่ใช่รายคน คิวยาวไป ตั้ง CHALLENGE_TIME=120 ทั้งสองเทอร์มินัล

เลือก seed

ใช้จาก seed_list.py (ทางสั้นยากอยู่ทางที่ pilot เดิมเลือก) เช่น 5, 7, 11 ทีมที่ทำขั้น 3 ได้จะถึงเส้นชัย ทีมที่ไม่ได้ทำจะไม่ถึง อยากให้ทุกทีมถึงเส้นชัย ใช้ 13

เฉลย

Instructor/answers/w7_challenge: pilot_fall_memory.py (ขั้น 3), pilot_ramp_check.py และ pilot_spikes.py (ขั้น 4) ไม่แจกนักเรียน